

Dizionario

Dere ore : *spinta*

Implementazione
Array
 $\Theta(1)$

Implementazione
Liste
 $\Theta(1)$

Array
Ordinato
 $O(n)$

Liste
ordinate
 $O(n)$

concellazione

$\Theta(n)$

$O(n)$

$O(n)$

$O(n)$

ricerca

$O(n)$

$O(n)$

$O(\lg(n))$ *Ricerca
binaria*

$O(n)$

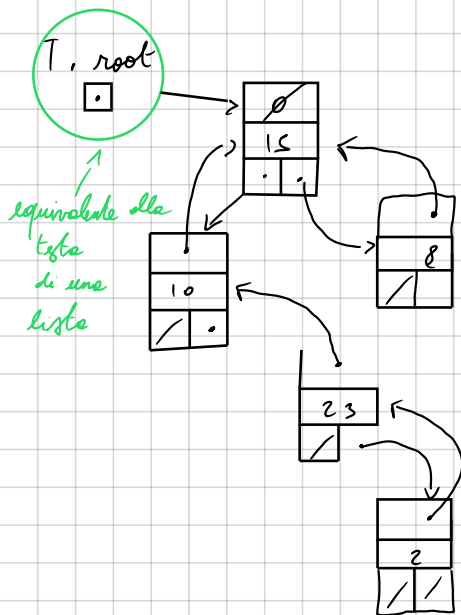
Per cercare di avere dei tempi migliori tocca usare gli **ABR** (*Alberi Binari di ricerca*)

ALB BINARIO

θ è un insieme vuoto o una tripla (K, x, dx) $K = \text{chiave}$

$x, dx = 2$ *AB disgiunti
definizione ricorsiva*

Esempio



↓

Nodo = $\text{key} : \text{INT}$
 $x : \text{Nodo}$
 $dx : \text{Nodo}$

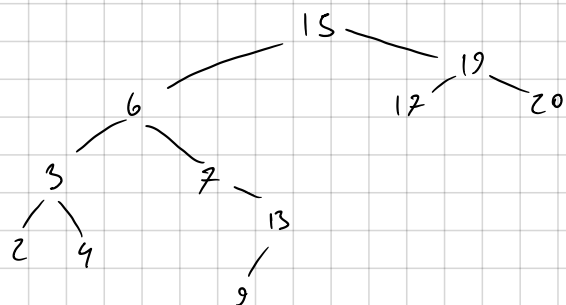
*Rappresentazione in
memoria*

Parent	
key	
x	dx

Un albero non avendo alcune proprietà di struttura rende difficile la ricerca, quindi dobbiamo usare un **AI**

T è un albero il quale $\forall$ nodo $x \in T$ se y è un nodo del sottoalbero di x allora $y.Key < x.Key$
 se y è un nodo del sottoalbero di x allora $y.Key > x.Key$

Esempio



BST_SEARCH (nodo, x)

BST_Search (T.root, x)

if nodo == nil then

return nil

if nodo.Key == Key then

return nodo

if $x < \text{nodo.Key}$ then

return BST_Search (nodo.left, x)

else

return BST_Search (nodo.right, x)

Tempi: $O(h)$

h è l'altezza dell'albero

$h = \Omega(\log(n)) / O(n)$

